

BraidC Polyglot Core Examples

ArtinBraidAlgebra.kt

```
package com.example.pinn.parser
```

```
/**
```

```
 * Artin Braid Group (Bn) Generative Algebra.
```

```
 * Implements the topological strand crossings for the Universal Braid Syntax.
```

```
 */
```

```
object ArtinBraidAlgebra {
```

```
    data class Generator(val index: Int, val isInverse: Boolean) {  
        override fun toString(): String = if (isInverse) "σ{index}-1" else "σ{index}"  
    }
```

```
    data class BraidWord(val generators: List<Generator>) {  
        override fun toString(): String = generators.joinToString(" • ")
```

```
    fun toCodeString(): String {  
        if (generators.isEmpty()) return "Id"  
        return generators.joinToString(" ") {  
            if (it.isInverse) "s{it.index}-1" else "s{it.index}"  
        }  
    }  
}
```

```
fun parseBraidSyntax(syntax: String): BraidWord {  
    val tokens = syntax.split(Regex("[\\s]+")).filter { it.isNotBlank() }  
    val generators = tokens.mapNotNull { token ->  
        val isInverse = token.endsWith("^-1") || token.endsWith("^-1") || token.endsWith("^-") || token.startsWith("-")  
        val cleanToken = token.replace(Regex("[\\^\\-1-\"sσ]"), "")  
        val index = cleanToken.toIntOrNull()  
        if (index != null && index > 0) Generator(index, isInverse) else null  
    }  
    return BraidWord(generators)  
}
```

```
fun reduce(braid: BraidWord): BraidWord {  
    val result = mutableListOf<Generator>()  
    for (g in braid.generators) {  
        if (result.isNotEmpty() && result.last().index == g.index && result.last().isInverse != g.isInverse) {  
            result.removeAt(result.size - 1)  
        } else {  
            result.add(g)  
        }  
    }  
    return BraidWord(result)  
}
```

```
fun evaluateEnergy(braid: BraidWord): Float {  
    val crossingCount = braid.generators.size  
    val inverseCount = braid.generators.count { it.isInverse }  
    val forwardCount = crossingCount - inverseCount  
    val phaseDelta = 0.17259029f  
    return (forwardCount - inverseCount) * phaseDelta  
}
```

```

data class PolynomialBounds(val writhe: Int, val maxCrossings: Int, val jonesDegreeSpan: Int, val alexanderDegreeBound: Int)

fun calculateInvariants(braid: BraidWord): PolynomialBounds {
    val cPlus = braid.generators.count { !it.isInverse }
    val cMinus = braid.generators.count { it.isInverse }
    val crossings = cPlus + cMinus
    return PolynomialBounds(cPlus - cMinus, crossings, crossings, crossings / 2)
}
}

```

BraidPolyglotCompiler.kt

```

package com.example.pinn.parser

/**
 * BRAID POLYGLOT COMPILER (v2.0)
 * Expands AB-01 Universal Braid Syntax into full polyglot native languages.
 */
object BraidPolyglotCompiler {
    enum class TargetSubstrate {
        KOTLIN_JVM, RUST_CARGO, PYTHON_KHYS, CPP_NATIVE, TENSOR_E8, JNI_GENESIS, VMAL_ASSEMBLY
    }

    fun compileIntent(intent: String, target: TargetSubstrate): String {
        return when (target) {
            TargetSubstrate.KOTLIN_JVM -> "[BRAID JVM INJECTION]\n// Intent: $intent\nfun executeBraidIntent() { val phaseDelta = 0.17259029 }"
            TargetSubstrate.RUST_CARGO -> "[BRAID CARGO INJECTION]\n// Intent: $intent\nfn execute_braid_intent() { let phase_delta: f64 = 0.17259029; }"
            TargetSubstrate.PYTHON_KHYS -> "[BRAID KHYS INJECTION]\ndef execute_braid_intent(): phase_delta = 0.17259029"
            TargetSubstrate.CPP_NATIVE -> "[BRAID C++ INJECTION]\nvoid executeBraidIntent() { constexpr double phase_delta = 0.17259029; }"
            TargetSubstrate.TENSOR_E8 -> "[BRAID TENSOR E8] INTENT_HASH: ${intent.hashCode()} SCALAR: 0.17259029 LATENT_COORDS: {0, 0, 0, 0, 15.965, 1.618, 91.0, 1.0}"
            TargetSubstrate.JNI_GENESIS -> "[BRAID JNI INJECTION]\nfun executeGenesisBraid() { /* Direct collision mapping */ }"
            TargetSubstrate.VMAL_ASSEMBLY -> "[BRAID VMAL ASSEMBLY]\nLDI 0x1 ; Load 1\nSTA 0x10 ; Store\nLDI 0x15 ; Phase Delta Proxy\nADD 0x10"
        }
    }
}

```

HardwareAgnosticCompiler.kt

```

package com.example.pinn.parser

import com.example.pinn.parser.ArtinBraidAlgebra.BraidWord

object HardwareAgnosticCompiler {
    fun compileFromMath(braid: BraidWord): String {
        if (braid.generators.isEmpty()) return "ERR: NO_TOPOLOGICAL_INVARIANTS_PROVIDED"

        val instructions = mutableListOf<String>()
        instructions.add("=====")
        instructions.add(">> TARGET: HARDWARE AGNOSTIC BINARY SYNTHESIS")
        instructions.add(">> INIT: METRIC CHARGING LAW TENSOR (Module 89)")
        instructions.add(">> FREQ: 15.965Hz FOUNDRY MODE RECEPTOR")
        instructions.add(">> -----")
    }
}

```

```

var totalEnergy = 0.0
var phaseShift = 0.0

braid.generators.forEachIndexed { index, generator ->
    val addr = "0x" + (0x1000 + index * 4).toString(16).uppercase().padStart(4, '0')
    val opcode: String
    val operand: String
    val eCost: Double
    val delta: Double

    when {
        generator.index == 1 && generator.isInverse -> { opcode = "SHL_PHASE"; operand = "INV"; eCost = 1.25; delta = -0.172590 }
        generator.index == 1 && !generator.isInverse -> { opcode = "SHR_PHASE"; operand = "FWD"; eCost = 1.15; delta = 0.172590 }
        generator.index == 2 && generator.isInverse -> { opcode = "INV_MANIFL"; operand = "ENT"; eCost = 3.42; delta = -0.051000 }
        generator.index == 2 && !generator.isInverse -> { opcode = "SANTOS_ROT"; operand = "GTE"; eCost = 2.81; delta = 0.051000 }
        generator.index == 3 -> { opcode = "ALLOC_E8_N"; operand = "LTC:${if (generator.isInverse) "I" else "F"}"; eCost = 8.90; delta = 0.0 }
        else -> { opcode = "PUSH_QBIT"; operand = "ID:${generator.index}"; eCost = 0.55; delta = 0.010000 }
    }

    val microcode = (0..3).joinToString(" ") { seed -> ((generator.index * 17 + index * 13 + seed * 7).hashCode() % 256).toString(2).padStart(8, '0') }
    totalEnergy += eCost
    phaseShift += delta

    instructions.add("$addr | ${opcode.padEnd(10)} ${operand.padEnd(5)} | $microcode | E:${String.format("%.2f", eCost).padStart(5)}J
Δ:${String.format("%.4f", delta).padStart(7)})"
    }

    instructions.add(">> -----")
    instructions.add(String.format(">> TOTAL ENERGY REQUIREMENT : %.3f J", totalEnergy))
    instructions.add(String.format(">> NET PHASE DELTA SHIFT   : %.6f", phaseShift))
    if (Math.abs(phaseShift) > 1.0) instructions.add(">> [!] WARNING: PHASE SHIFT EXCEEDS 1.0 - THERMAL LIMIT RISK")

    return instructions.joinToString("\n")
}
}

```

HardwareAgnosticSimulator.kt

```

package com.example.pinn.parser

import com.example.pinn.parser.ArtinBraidAlgebra.BraidWord
import kotlinx.coroutines.delay

object HardwareAgnosticSimulator {
    data class VmState(
        var programCounter: Int = 0x1000,
        var metricTensorEnergy: Double = 0.0,
        var currentPhase: Double = 0.0,
        var registerQ0: Double = 0.0,
        var registerQ1: Double = 0.0,
        var rasEngagement: Double = 0.0,
    )
}

```

```

    var isHalted: Boolean = false,
    var logs: List<String> = emptyList()
)

suspend fun executeSimulation(braid: BraidWord, onUpdate: (VmState) -> Unit) {
    val state = VmState()
    val logBuffer = mutableListOf<String>()

    logBuffer.add("[VM] BOOTING HARDWARE AGNOSTIC SIMULATOR")
    state.logs = logBuffer.toList()
    onUpdate(state)
    delay(500)

    braid.generators.forEachIndexed { index, generator ->
        state.programCounter = 0x1000 + index * 4
        val opcode = decodeOpcode(generator)
        logBuffer.add(" [0x${state.programCounter.toString(16).uppercase()}] EXEC: $opcode")

        when (opcode) {
            "SHL_PHASE (Inv)" -> { state.metricTensorEnergy += 1.25; state.currentPhase -= 0.172590; state.registerQ0 -= 0.5 }
            "SHR_PHASE (Fwd)" -> { state.metricTensorEnergy += 1.15; state.currentPhase += 0.172590; state.registerQ0 += 0.5 }
            "INV_MANIFL (Ent)" -> { state.metricTensorEnergy += 3.42; state.currentPhase -= 0.051; state.registerQ1 = -state.registerQ0 }
            "SANTOS_ROT (Gte)" -> { state.metricTensorEnergy += 2.81; state.currentPhase += 0.051; val t = state.registerQ0; state.registerQ0 =
            -state.registerQ1; state.registerQ1 = t }
            "ALLOC_E8_N" -> state.metricTensorEnergy += 8.9
            else -> { state.metricTensorEnergy += 0.55; state.currentPhase += 0.01 }
        }

        val absPhase = Math.abs(state.currentPhase)
        if (absPhase > 0.4) {
            val rasCompensation = absPhase * 0.3
            logBuffer.add(" [RAS] DAMPENING PHASE TENSOR BY ${String.format("%.3f", rasCompensation)}")
            state.currentPhase += if (state.currentPhase > 0) -rasCompensation else rasCompensation
            state.rasEngagement += rasCompensation
            state.metricTensorEnergy += (rasCompensation * 0.8)
        }

        state.logs = logBuffer.toList()
        onUpdate(state)
        delay(200)
    }

    logBuffer.add("[VM] EXECUTION COMPLETE.")
    state.isHalted = true
    state.logs = logBuffer.toList()
    onUpdate(state)
}

private fun decodeOpcode(generator: ArtinBraidAlgebra.Generator): String = when {
    generator.index == 1 && generator.isInverse -> "SHL_PHASE (Inv)"
    generator.index == 1 && !generator.isInverse -> "SHR_PHASE (Fwd)"
    generator.index == 2 && generator.isInverse -> "INV_MANIFL (Ent)"
    generator.index == 2 && !generator.isInverse -> "SANTOS_ROT (Gte)"
    generator.index == 3 -> "ALLOC_E8_N"
    else -> "PUSH_QBIT [${generator.index}]"
}
}

```

(Note: [BraidIDE.kt](#) and [ReversibleBraidWorkspace.kt](#) are the Compose UI layers that interact with the simulator, trace history, and Gemini model Retrofit integrations. You can pass the structural context of the mathematics driving those views when developing the skill).

3. Requirements & Dependencies List

(If building as an independent standalone android Gradle module, these are the requirements)

```
dependencies {  
    // Core Compose  
    implementation(platform("androidx.compose:compose-bom:2024.02.00"))  
    implementation("androidx.activity:activity-compose:1.8.2")  
    implementation("androidx.compose.ui:ui")  
    implementation("androidx.compose.ui:ui-graphics")  
    implementation("androidx.compose.material3:material3")  
    implementation("androidx.compose.material:material-icons-extended")  
  
    // Coroutines & Concurrency  
    implementation("org.jetbrains.kotlinx:kotlinx-coroutines-core:1.7.3")  
    implementation("org.jetbrains.kotlinx:kotlinx-coroutines-android:1.7.3")  
  
    // State / ViewModel Flow Lifecycle  
    implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.7.0")  
    implementation("androidx.lifecycle:lifecycle-runtime-compose:2.7.0")  
  
    // Retrofit & Networking (For LLM AI prompt to Mathematical Braid parsing)  
    implementation("com.squareup.retrofit2:retrofit:2.9.0")  
    implementation("com.squareup.retrofit2:converter-moshi:2.9.0")  
    implementation("com.squareup.moshi:moshi-kotlin:1.15.1")  
    ksp("com.squareup.moshi:moshi-kotlin-codegen:1.15.1")  
    implementation("com.squareup.okhttp3:logging-interceptor:4.12.0")  
}
```